UNIVERSITAS PAHLAWAN TUANKU TAMBUSAI
FAKULTAS TEKNIK — PROGRAM STUDI TEKNIK INFORMATIKA

PANDUAN PRAKTIKUM PERTEMUAN 4

Milik Kelas dan Milik Objek: Konstanta, classmethod, dan staticmethod

Mata Kuliah	Pemrograman Berorientasi Objek · 4 sks · Semester III
Sub-CPMK	Sub-CPMK-4 — Menggunakan tipe data bawaan, f-string, konstanta Final, serta membedakan anggota milik kelas dan milik instans termasuk classmethod dan staticmethod.
Bobot terhadap nilai akhir	5% — termasuk Tugas 1 (individu)
Alokasi waktu	Terbimbing 270 menit (kuliah 100 + praktikum 170) · Penugasan terstruktur 205 menit · Belajar mandiri 200 menit
Profil gerbang	p04 — minimal dua kelas, dan minimal satu konstanta tingkat kelas
Bekal dari Pertemuan 3	Repositori bertanda versi p03 dengan property, setter bervalidasi, dan objek nilai Alamat.
Luaran pertemuan	AlatTani dengan konstanta bernama, pencacah milik kelas, konstruktor alternatif, dan dua method tanpa objek. Bertanda versi p04.

1. UTANG YANG KITA TINGGALKAN MINGGU LALU

Pada Pertemuan 3 kita menulis setter seperti ini:

```
        if nilai <= 0:
            raise ValueError(...)
        if nilai > 5_000_000:            ←  angka ini dari mana?
            raise ValueError(...)
```

Angka 5.000.000 muncul begitu saja tanpa nama dan tanpa penjelasan. Kalau esok hari pengurus UPJA memutuskan batas atasnya berubah, siapa yang tahu di berkas mana angka itu bersembunyi? Angka semacam ini disebut angka ajaib, dan pertemuan ini menaikkannya menjadi konstanta bernama.

2. TUJUAN DAN INDIKATOR

Setelah menyelesaikan pertemuan ini Anda mampu:

1. Mengubah angka ajaib menjadi konstanta tingkat kelas beranotasi Final.
2. Membedakan anggota milik kelas dan milik objek, serta menjelaskan akibat bila keliru menempatkannya.
3. Membuat konstruktor alternatif dengan @classmethod sebagai jawaban Python atas ketiadaan overloading.
4. Membuat @staticmethod untuk aturan yang dapat ditanyakan tanpa membuat objek terlebih dahulu.
5. Menjelaskan mengapa parameter berdefault yang dapat diubah berbahaya, dan mengenali gejalanya.
6. Memakai f-string untuk perataan dan pemformatan angka.

3. EMPAT JENIS ANGGOTA KELAS

Sampai Pertemuan 3 seluruh anggota kelas Anda adalah method biasa yang menerima self. Mulai sekarang ada empat jenis, dan memilih yang tepat adalah keterampilan yang dinilai.

Jenis	Parameter Pertama	Dapat Menyentuh	Dipakai Ketika
Konstanta kelas	–	Dibaca siapa pun	Aturan usaha yang berlaku sama bagi seluruh objek. TARIF_MINIMUM.

Jenis	Parameter Pertama	Dapat Menyentuh	Dipakai Ketika
Atribut kelas	—	Dibagi seluruh objek	Data yang memang milik bersama, misalnya pencacah alat terdaftar.
Method biasa	`self`	Data objek ini	Perilaku yang bergantung pada isi objek. biaya_sewa().
@classmethod	`cls`	Kelas dan konstantanya	Konstruktor alternatif, atau membaca data milik kelas. dari_baris_csv().
@staticmethod	—	Tidak keduanya	Aturan yang berkaitan dengan kelas tetapi tidak perlu objek. tarif_wajar().

4. ALUR PERTEMUAN

Waktu	Kegiatan	Bentuk
0–20′	Pembukaan, pembahasan tugas Petugas, dan kuis awal	Kuliah interaktif; LKM Bagian A
20–65′	Blok 1: angka ajaib, konstanta, dan milik siapa sebuah data	Kuliah dan live coding
65–105′	Blok 2: classmethod, staticmethod, dan f-string	Kuliah dan demonstrasi
105–115′	Istirahat	—
115–285′	Blok 3: praktikum terbimbing, Langkah 1 sampai 10	Praktikum; LKM Bagian B
285–305′	Blok 4: aktivitas kelompok menempatkan anggota kelas	Diskusi kelompok; LKM Bagian C
305–320′	Blok 5: penutup dan penjelasan Tugas 1	Refleksi; LKM Bagian D

5. LANGKAH PRAKTIKUM

Kerjakan berurutan pada repositori yang sama. Sebagian besar pekerjaan hari ini adalah merapikan kelas AlatTani yang sudah ada.

1
10′

Menyiapkan repositori

```
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # Linux atau macOS
git pull

python periksa_pbo.py src --profil p03
python -m pytest
```

2
15′

Menaikkan angka ajaib menjadi konstanta

Tambahkan blok konstanta di bagian paling atas badan kelas AlatTani, tepat di bawah docstring-nya. Impor Final dari typing.

```
from typing import Final


class AlatTani:
    """Satu unit alat pertanian beserta tarif hariannya."""

    # --- Aturan usaha UPJA: berlaku sama bagi seluruh alat ---
    TARIF_MINIMUM: Final[int] = 10_000
    TARIF_MAKSIMUM: Final[int] = 5_000_000
    PERSEN_DENDA_HARIAN: Final[int] = 10
    PEMISAH_BARIS: Final[str] = ";"
```

Tiga hal yang dituntut: nama HURUF_BESAR karena ini konstanta, anotasi Final agar mypy menolak siapa pun yang mencoba mengubahnya, dan garis bawah pada 10_000 yang tidak mengubah nilai tetapi membuat angka lebih mudah dibaca manusia.

3
20′

Membuat aturan yang dapat ditanya tanpa objek

Kewajaran tarif adalah aturan milik kelas, bukan milik satu alat tertentu. Pengurus UPJA seharusnya bisa bertanya “apakah enam juta wajar?” tanpa perlu mendaftarkan alat lebih dulu. Itulah gunanya @staticmethod.

```
    @staticmethod
    def tarif_wajar(nilai: int) -> bool:
        """Memeriksa kewajaran tarif tanpa perlu membuat alat lebih dulu."""
        return AlatTani.TARIF_MINIMUM <= nilai <= AlatTani.TARIF_MAKSIMUM

    @staticmethod
    def format_rupiah(nilai: int) -> str:
        """Mengubah angka menjadi tulisan rupiah bergaya Indonesia."""
        return "Rp" + f"{nilai:,}".replace(",", ".")
```

Lalu sederhanakan setter tarif agar memanggil aturan itu, bukan mengulang perbandingannya:

```
    @tarif_harian.setter
    def tarif_harian(self, nilai: int) -> None:
        if not AlatTani.tarif_wajar(nilai):
            raise ValueError(
                f"tarif harus antara "
                f"{AlatTani.format_rupiah(AlatTani.TARIF_MINIMUM)} dan "
                f"{AlatTani.format_rupiah(AlatTani.TARIF_MAKSIMUM)}, "
                f"diterima {nilai}"
            )
        self._tarif_harian = nilai
```

Perhatikan bahwa pesan galatnya kini menyebut batas yang berlaku, bukan sekadar menolak. Uji sendiri: python -c "from src.model.alat_tani import AlatTani; print(AlatTani.tarif_wajar(6_000_000))" — berjalan tanpa satu pun objek dibuat.

4
20′

Menambahkan pencacah milik kelas

UPJA ingin tahu berapa alat yang sudah terdaftar. Angka itu bukan milik satu alat mana pun — ia milik kelasnya. Tambahkan atribut kelas dan naikkan di konstruktor.

```
    # --- Milik kelas, bukan milik objek: dibagi seluruh alat ---
    _jumlah_terdaftar: int = 0

    def __init__(self, kode: str, nama: str, tarif_harian: int) -> None:
        ...
        self.tarif_harian = tarif_harian
        AlatTani._jumlah_terdaftar += 1        # ← nama kelas, bukan self

    @classmethod
    def jumlah_terdaftar(cls) -> int:
        """Banyaknya alat yang sudah pernah dibuat."""
        return cls._jumlah_terdaftar
```

Perhatikan baik-baik baris yang ditandai. Ia menyebut AlatTani, bukan self. Langkah berikutnya menjelaskan mengapa perbedaan itu menentukan segalanya.

5
15′

Kesalahan yang disengaja ①: pencacah yang tidak pernah bergerak

Buat berkas sementara pencacah_bocor.py, lalu jalankan.

```
class AlatTani:
    """Alat pertanian dengan pencacah yang tidak pernah bekerja."""

    _jumlah_terdaftar: int = 0

    def __init__(self, kode: str) -> None:
        self._kode = kode
        self._jumlah_terdaftar += 1        # SALAH: self, bukan AlatTani

    def laporan(self) -> str:
        return f"{self._kode} melaporkan jumlah = {self._jumlah_terdaftar}"
```

Keluaran setelah membuat tiga alat:

```
TR-01 melaporkan jumlah = 1
TR-02 melaporkan jumlah = 1
PA-01 melaporkan jumlah = 1
Pencacah milik kelas : 0
Isi objek a          : ['_kode', '_jumlah_terdaftar']
```

Yang sebenarnya terjadi. Baris self._jumlah_terdaftar += 1 dibaca Python sebagai self._jumlah_terdaftar = self._jumlah_terdaftar + 1. Sisi kanan membaca dari kelas dan menemukan nol; sisi kiri MENULIS ke objek, sehingga lahirlah atribut baru milik objek yang menutupi atribut kelas. Setiap alat mengaku dirinya alat pertama, dan pencacah kelas tidak pernah bergerak. Perhatikan baris terakhir keluaran: vars() membuktikan atribut itu kini menempel pada objek. Catat pada LKM Bagian B.1.

6
25′

Membuat konstruktor alternatif

Pada Pertemuan 2 kita menyinggung bahwa Python tidak menyediakan overloading konstruktor. Inilah jawabannya. Data alat UPJA datang dalam bentuk baris pendataan seperti “TR-01;Traktor;150000”, dan kita ingin membuat objek langsung dari baris itu.

```
    @classmethod
    def dari_baris_csv(cls, baris: str) -> "AlatTani":
        """Membuat alat dari satu baris pendataan."""
        bagian = baris.strip().split(cls.PEMISAH_BARIS)
        if len(bagian) != 3:
            raise ValueError(
                f"baris harus berisi 3 bagian, diterima {len(bagian)}: '{baris}'")
        kode, nama, tarif = bagian
        if not tarif.strip().isdigit():
            raise ValueError(f"tarif harus berupa angka, diterima '{tarif}'")
        return cls(kode.strip(), nama.strip(), int(tarif))
```

Tiga hal yang perlu Anda perhatikan. Pertama, method ini menerima cls, bukan self — masuk akal, karena saat dipanggil belum ada objek apa pun. Kedua, baris terakhir memanggil cls(…) yang berarti memanggil konstruktor biasa, sehingga seluruh validasi Pertemuan 3 tetap berjalan. Ketiga, tipe kembaliannya ditulis dalam tanda kutip karena kelasnya belum selesai didefinisikan pada saat baris itu dibaca.

```
alat = AlatTani.dari_baris_csv("PA-01;Pompa Air 3 Inci;75000")
```

7
15′

Kesalahan yang disengaja ②: daftar yang dibagi bersama

Buat berkas sementara default_berbagi.py, lalu jalankan.

```
class Penyewa:
    """Penyewa dengan riwayat sewa yang tanpa sengaja dibagi bersama."""

    def __init__(self, nama: str, riwayat: list[str] = []) -> None:   # SALAH
        self._nama = nama
        self._riwayat = riwayat

    def sewa(self, kode_alat: str) -> None:
        self._riwayat.append(kode_alat)


budi = Penyewa("Budi")
siti = Penyewa("Siti")
budi.sewa("TR-01")
budi.sewa("PA-01")
```

Keluaran:

```
Budi: ['TR-01', 'PA-01']
Siti: ['TR-01', 'PA-01']          ←  Siti belum menyewa apa pun
Daftar yang sama? True
```

Nilai default dibuat sekali saja. Daftar kosong itu dibuat satu kali ketika baris def dijalankan, bukan setiap kali objek dibuat. Seluruh penyewa yang tidak mengisi parameter tersebut berbagi daftar yang sama persis. Perbaikannya: tulis riwayat: list[str] | None = None, lalu di dalam konstruktor tulis self._riwayat = list(riwayat) if riwayat else []. Ini pembayaran janji dari Pertemuan 2, ketika kita hanya berkata “jangan pakai kurung siku sebagai default, alasannya nanti”.

Sekarang jalankan Ruff terhadap kedua berkas kesalahan tadi, dan bandingkan hasilnya:

```
python -m ruff check default_berbagi.py
→ B006 Do not use mutable data structures for argument defaults

python -m ruff check pencacah_bocor.py
→ All checks passed!
```

Perhatikan perbedaannya. Kesalahan kedua tertangkap alat; kesalahan pertama lolos dari semua alat yang kita punya. Tidak ada perkakas yang menangkap segalanya. Itulah sebabnya pemahaman dan pengujian tetap tidak tergantikan.

8
15′

Denda keterlambatan dan pemformatan angka

Tambahkan satu method yang memakai konstanta persen denda, lalu rapikan keluaran dengan f-string.

```
    def denda_keterlambatan(self, hari_terlambat: int) -> int:
        """Menghitung denda keterlambatan pengembalian alat."""
        if hari_terlambat < 0:
            raise ValueError(
                f"hari terlambat tidak boleh minus, diterima {hari_terlambat}")
        per_hari = self._tarif_harian * AlatTani.PERSEN_DENDA_HARIAN // 100
        return per_hari * hari_terlambat
```

Lalu pada src/main.py, gunakan perataan f-string agar daftar tarif tersusun rapi:

```
for calon in (5_000, 150_000, 6_000_000):
    status = "wajar" if AlatTani.tarif_wajar(calon) else "ditolak"
    print(f"  {AlatTani.format_rupiah(calon):>14}  →  {status}")
```

Tanda :>14 berarti ratakan ke kanan selebar empat belas karakter. Bandingkan keluarannya dengan tanpa perataan — perbedaannya kecil di layar, tetapi besar artinya bagi pengurus UPJA yang membaca laporan.

9
20′

Menulis pengujian

Minimal dua puluh satu fungsi uji seluruhnya. Yang baru pada pertemuan ini adalah pengujian terhadap anggota kelas dan konstruktor alternatif.

```
def test_tarif_wajar_dipanggil_tanpa_objek() -> None:
    assert AlatTani.tarif_wajar(150_000) is True
    assert AlatTani.tarif_wajar(6_000_000) is False


def test_pencacah_naik_untuk_setiap_alat_baru() -> None:
    awal = AlatTani.jumlah_terdaftar()
    AlatTani("TR-90", "Traktor Uji", 100_000)
    AlatTani("TR-91", "Traktor Uji", 100_000)
    assert AlatTani.jumlah_terdaftar() == awal + 2


def test_konstruktor_dari_baris_csv() -> None:
    alat = AlatTani.dari_baris_csv("PA-01;Pompa Air;75000")
    assert alat.kode == "PA-01"
    assert alat.tarif_harian == 75_000


def test_baris_csv_tarif_bukan_angka_ditolak() -> None:
    with pytest.raises(ValueError):
        AlatTani.dari_baris_csv("PA-01;Pompa Air;tujuh puluh lima ribu")
```

Perhatikan uji pencacah. Ia tidak menuliskan angka pasti seperti assert jumlah == 2, melainkan menyimpan nilai awal lalu memeriksa selisihnya. Alasannya: uji lain mungkin sudah membuat alat lebih dulu, dan urutan pelaksanaan uji tidak dijamin. Uji yang bergantung pada urutan adalah uji yang rapuh.

10
15′

Menjalankan ketiga gerbang dan mengumpulkan

```
python periksa_pbo.py src --profil p04
python -m mypy
python -m pytest
python -m src.main
```

Keluaran yang diharapkan:

```
LOLOS. Tidak ada pelanggaran disiplin berorientasi objek.
Success: no issues found in 10 source files
21 passed

Sebelum pendataan, alat terdaftar: 0
  [TR-01] Traktor Roda Dua Kubota — Rp150.000/hari
  [PA-01] Pompa Air 3 Inci — Rp75.000/hari
Sesudah pendataan, alat terdaftar: 4

--- Memeriksa tarif tanpa membuat objek ---
         Rp5.000  →  ditolak
       Rp150.000  →  wajar
     Rp6.000.000  →  ditolak
```

Hapus pencacah_bocor.py dan default_berbagi.py, lalu simpan pekerjaan Anda.

```
git add .
git commit -m "P04: konstanta kelas, pencacah, konstruktor alternatif"
git tag p04
git push origin main --tags
```

6. AKTIVITAS KELOMPOK: MENEMPATKAN ANGGOTA KELAS

Berkelompok, 20 menit. Kembali ke kelas Keramba milik kelompok Anda.

Untuk setiap kebutuhan di bawah ini, tentukan jenis anggota yang tepat — konstanta kelas, atribut kelas, method biasa, classmethod, atau staticmethod — beserta alasannya. Isi pada LKM Bagian C.

Kebutuhan	Pertanyaan Penuntun
Tingkat kelulusan hidup baku 80%	Apakah angka ini berbeda untuk tiap keramba?
Berat panen sebuah keramba	Perlu tahu isi keramba yang mana?
Jumlah keramba yang terdaftar	Milik keramba mana angka ini?
Membuat keramba dari baris pendataan	Sudah ada objeknya ketika method ini dipanggil?
Memeriksa apakah sebuah luas masuk akal	Perlu keramba tertentu untuk menjawabnya?
Luas minimum keramba yang diizinkan	Berubah dari waktu ke waktu, atau tetap?

Satu butir tambahan bernilai paling tinggi: temukan satu kebutuhan yang menurut kelompok Anda tidak layak menjadi anggota kelas Keramba sama sekali — melainkan seharusnya menjadi milik kelas lain — beserta alasannya.

7. TUGAS 1 (INDIVIDU) — DINILAI TERPISAH

Perhatian. Berbeda dari penugasan pertemuan sebelumnya, Tugas 1 menyumbang bagian tersendiri pada nilai akhir mata kuliah. Kerjakan sendiri; kemiripan kode antar mahasiswa akan ditelusuri melalui riwayat commit.

Tugas	Buat kelas Kios untuk sebuah UMKM di Bangkinang yang menjual hasil pertanian: beras, gula, minyak goreng, dan sejenisnya.
Konstanta kelas	Minimal tiga, beranotasi Final. Salah satunya wajib berupa persentase yang dipakai dalam perhitungan, misalnya margin keuntungan minimum.
Atribut kelas	Satu pencacah yang menghitung banyaknya produk terdaftar, dinaikkan dengan benar melalui nama kelas.
classmethod	Satu konstruktor alternatif yang membangun objek dari baris pendataan berformat teks.
staticmethod	Satu aturan yang dapat ditanyakan tanpa membuat objek, misalnya kewajaran sebuah harga jual.
Pemformatan	Minimal satu keluaran memakai perataan f-string sehingga daftar tersusun rapi.
Pengujian	Minimal sepuluh fungsi uji, minimal lima membuktikan penolakan. Uji pencacah wajib memakai pola selisih, bukan angka pasti.
Gerbang	Wajib lolos periksa_pbo.py --profil p04, mypy, dan pytest. Tanda versi p04-tugas1.
Tenggat	Pukul 23.59 pada hari sebelum Pertemuan 5.
Belajar mandiri	Rujukan [R1] Bab 2 dan 6, [R2] Bab 2–3, [R4] Bab 8–12.

8. DAFTAR PERIKSA SEBELUM MENINGGALKAN LABORATORIUM

☐	Minimal empat konstanta kelas beranotasi Final sudah ada di AlatTani.
☐	Setter tarif memanggil AlatTani.tarif_wajar(), bukan mengulang perbandingan.
☐	Pencacah dinaikkan dengan AlatTani._jumlah_terdaftar, bukan self.
☐	Pengamatan pencacah yang tidak bergerak sudah dicatat pada LKM Bagian B.1.
☐	Pengamatan daftar yang dibagi bersama sudah dicatat pada LKM Bagian B.2.
☐	Hasil perbandingan Ruff terhadap kedua berkas kesalahan sudah dicatat.
☐	Konstruktor alternatif dari_baris_csv berjalan dan menolak baris cacat.

☐	Minimal 21 fungsi uji; uji pencacah memakai pola selisih.
☐	Ketiga gerbang hijau dengan profil p04; kedua berkas kesalahan sudah dihapus.
☐	Tanda versi p04 sudah terdorong ke GitHub.